

VOLUME 06 / RUST BACKEND MASTERY

Web 基礎と Topcoat 0.10 完全ガイド

HTTP の要求と応答から、Topcoat のページ・ルーター・view!・フォーム・アセット・セッションまでを、動かして確かめる

第6巻: Web 基礎と Topcoat 0.10 完全ガイド

この巻で身に付けること

第5巻までで、Rust の文法・所有権・設計・async を学びました。この巻では、それを使って **Web のサーバー**を作ります。ブラウザから届いた「この画面を見せて」という頼みに、Rust のプログラムが HTML を作って返す仕組みです。

Web のサーバーは、いきなり道具（フレームワーク）から始めると、何が起きているのかが見えにくくなります。次のような場面で、困ることがよくあります。

- ブラウザでは何も表示されないのに、どこが悪いのか見当がつかない
- 「404」「405」「303」のような数字が返ってきたが、それが何を意味するのか分からない
- フォームを送ったあとに再読み込みしたら、同じデータがもう1件できてしまった
- 日本語の文字が、途中で切れて表示される

そこで、この巻の最初の単元では、道具を使わずに、**標準ライブラリだけ**で小さなサーバーを書き、HTTP の要求と応答が「ただの文字のやり取り」であることを、自分の目で確かめます。そのうえで、UNIT 02 から、RustNote でも使っている Web フレームワーク **Topcoat 0.10** の使い方を、1つずつ学びます。

到達目標

この巻を終えると、次のことができるようになります。

- HTTP の要求（メソッド・パス・ヘッダー・本文）と応答（状態コード・ヘッダー・本文）の形を説明し、`curl.exe -i` や `-v` の表示を読める
- 状態コード（200・303・403・404・405 など）と、GET・POST の使い分けを説明できる
- Topcoat 0.10 で、ページ（`#[page]`）・ルーター・レイアウト・URL の部品の読み取り・`view!` とコンポーネントを使った画面を作れる
- フォームを受け取り、成功したら 303 で GET の画面へ移す（PRG）形を作れる
- 共有の状態（`app_context`）・アセット（CSS の束）・エラー・クッキー・セッション・ほかのサイトからの送信の守りを、Topcoat の仕組みで扱える

学習地図

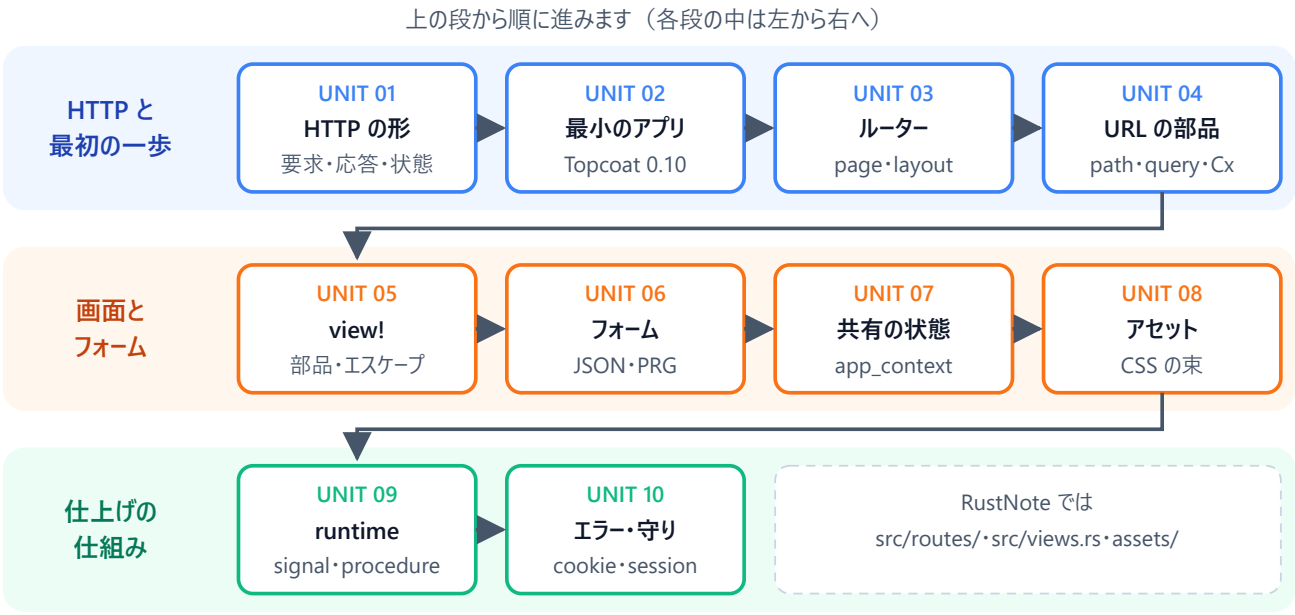


図6-1: 第6巻の学習地図。道具を使わずに HTTP の形を確かめてから、Topcoat で画面とフォームを作り、共有の状態・アセット・エラーと守りの仕組みで仕上げる

単元	テーマ	ひとことで言うと
UNIT 01	HTTP の要求・応答・状態コード	標準ライブラリだけのサーバーで、HTTP がただの文字だと確かめる
UNIT 02	Topcoat 0.10 の最小のアプリ（準備中）	ページを1つ作って起動する
UNIT 03	ルーター・page・route・layout（準備中）	URL ごとに処理を分け、共通の外枠で包む
UNIT 04	URL の部品（path・query）と Cx（準備中）	`/notes/{id}` や `?page=2` を型にして読む
UNIT 05	view!・コンポーネント・HTML のエスケープ（準備中）	画面を部品に分け、入力を安全に表示する
UNIT 06	フォーム・JSON・PRG（準備中）	送られてきた値を型にし、303 で移る
UNIT 07	app_context・memoize（準備中）	DB などを全部の処理で共有し、同じ問い合わせを1回にする
UNIT 08	アセット・CSS・束（準備中）	`asset!` と `topcoat asset bundle`
UNIT 09	runtime・signal・procedure・shard（準備中）	ブラウザの中で動く部分と、その守り方
UNIT 10	エラー・クッキー・セッション・CSRF（準備中）	400・404・リダイレクトと、ログインの仕組みの土台

> **お知らせ:** この版には UNIT 01 を収録しています。UNIT 02〜10 は準備中で、次の更新から順に追加します。以前の版（Topcoat 0.5 の版）の PDF は、この版がそろうまで残してあります。Topcoat は 0.5 から 0.10 までに書き方が大きく変わっているので、以前の版のコードは、0.10 では動かないことがあります。

前提知識（先に読んでおく巻）

- **第2巻（Rust 基礎）**：`String` と `&str`、`match`、`Vec`、`for`。特に、日本語の文字列の「文字数」と「バイト数」の違い（第2巻 UNIT 05）。UNIT 01 のエラー1で、この違いが原因の事故を見ます。
- **第4巻（設計）**：`Result` と `?`、標準ライブラリの入出力（`std::io`）。
- **第5巻（async）**：UNIT 02 から使う `#[tokio::main]` と `async fn`。

この巻の進め方

各単元では、次の順番で進めてください。

1. **予想する:** コマンドを実行する前に、どの状態コードが返り、何が表示されるかを予想します。
2. **そのまま動かす:** サーバーを起動し、もう1つの PowerShell の窓から `curl.exe` で要求を送ります。
3. **両方の窓を見る:** サーバーの窓（届いた要求）と、curl の窓（返ってきた応答）を、並べて見比べます。
4. **1か所だけ変える:** 「よくあるエラーと読み方」の節のとおりに書き換えて、何が変わるかを見ます。

> **ヒント:** この巻では、サーバーを動かす窓と、`curl.exe` を打つ窓の2つの PowerShell を使います。サーバーを止めるときは、サーバーの窓で Ctrl+C を押します。止め忘れたまま、別のサーバーを同じポートで起動しようとすると、エラーになります。

使うツールと版

この巻のコードは、すべて次の環境で実際にコンパイル・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.98 以上**（本書シリーズの基準）。動作確認は rustc 1.99.0 / cargo 1.99.0（`aarch64-pc-windows-msvc`）
edition	2024
curl	8.21.0（Windows に入っている `curl.exe`。PowerShell では `curl` だけだと別のコマンドになるので、必ず `curl.exe` と打ちます）
topcoat	0.10.0（UNIT 02 から使います。UNIT 01 では使いません）

練習用のプロジェクトを用意する

UNIT 01 では、外のクレートを1つも使いません。練習用のプロジェクトを1つ作り、`src/bin/` に、単元ごとの小さなサーバーを足していきます。

```
cargo new http_lab
cd http_lab
# このフォルダでは Rust 1.99.0 を使う（第13巻 UNIT 01）
Set-Content rust-toolchain.toml "[toolchain]`nchannel = `"1.99.0`""
New-Item -ItemType Directory src\bin
```

`src/bin/u01_raw.rs` のように置いたファイルは、`cargo run --bin u01_raw` で、それぞれ別のプログラムとして動かせます。

UNIT 01: HTTP の要求・応答・状態コード

この単元のゴール

- HTTP の**要求**（要求行・ヘッダー・空行・本文）と**応答**（状態行・ヘッダー・空行・本文）の形を、実際に届いた文字で説明できる
- 標準ライブラリだけの小さなサーバーを動かし、`curl.exe -i` と `-v` の表示を読める
- 状態コード（200・301・303・403・404・405）が、それぞれ何を伝えるのかを説明できる
- GET（見るだけ）と POST（変える）の違いと、POST の後に 303 で GET の画面へ移す（PRG）理由を説明できる

たとえ話でつかむ

役所の窓口に、申請書を出す場面を考えます。

- **申請書**のいちばん上には、「何をしたいか」（閲覧したい・登録したい）と「どの書類について」（住民票・印鑑登録）を書きます。その下に、名前や連絡先のような**添え書き**を並べ、1行空けてから、**中身**（登録する内容）を書きます。
- 窓口の人は、申請書を読んで、**返事の紙**を返します。いちばん上には「受け付けました」「その書類はありません」「その窓口ではできません」のような**結果の番号**が書いてあり、その下に添え書き、1行空けて中身（書類の写しなど）が続きます。
- 「登録」の申請が終わったら、窓口の人は「控えは、隣の閲覧の窓口で受け取ってください」と案内します。登録の窓口に何度も並び直すと、同じ登録が二重にされてしまうからです。

Web のブラウザとサーバーのやり取り（HTTP）も、同じ形をしています。

役所の窓口	HTTP
申請書のいちばん上（何を・どの書類について）	**要求行**（メソッドとパス。`GET /hello HTTP/1.1`）
申請書の添え書き	**ヘッダー**（`Host:`・`User-Agent:`・`Content-Length:` など）
1行空けてから書く中身	**本文**（フォームの値など。GET には、ふつう無い）
返事の紙の結果の番号	**状態コード**（`200 OK`・`404 Not Found` など）
閲覧の窓口・登録の窓口	**GET**（見るだけ）・**POST**（変える）
「控えは閲覧の窓口で」	**303 See Other**（POST の後に、GET の画面へ移す。PRG）

> **用語:** HTTP（エイチティーティーピー。HyperText Transfer Protocol：ブラウザとサーバーが、要求と応答をやり取りするときの決まり）

> **用語:** 要求（ようきゅう。request：ブラウザなどからサーバーへの頼み。リクエストとも言う）／応答（おうとう。response：サーバーからの返事。レスポンスとも言う）

> **用語:** 状態コード（じょうたいコード。status code：応答の1行目にある3桁の数。結果の種類を表す。ステータスコードとも言う）

図で理解する

要求と応答は、どちらも「1行目・ヘッダー・空行・本文」の4つの部分でできています。

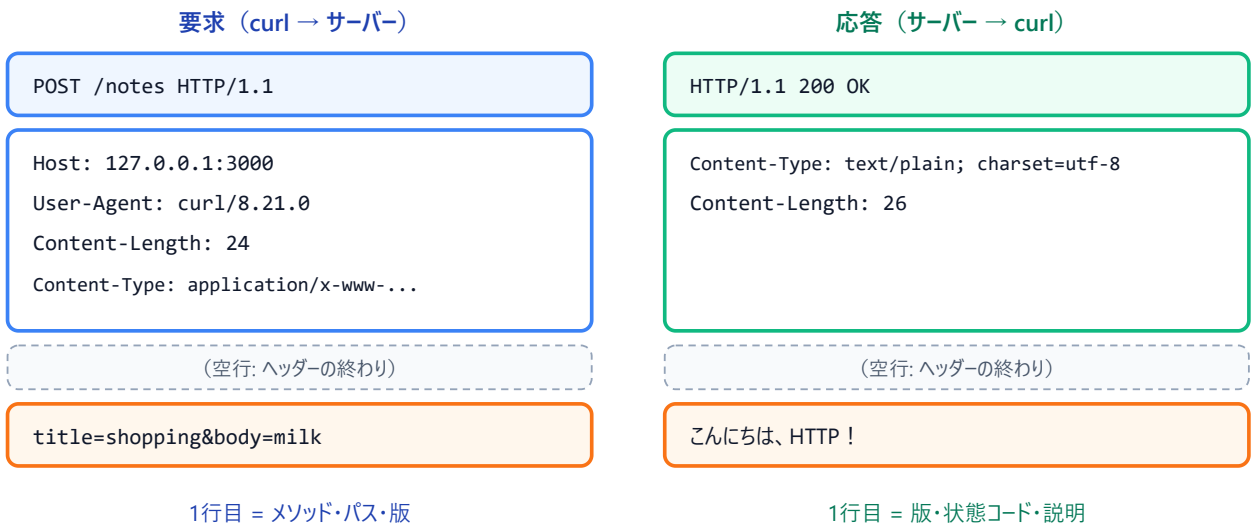


図6-2: HTTP の要求と応答の形。どちらも「1行目・ヘッダー・空行・本文」でできている。この単元の例で、実際に届いた・返した文字

- **1行目**だけが、要求と応答で違います。要求は「メソッド（`GET`・`POST` など）・パス（`/notes`）・版」、応答は「版・状態コード（`200`）・その説明（`OK`）」です。
- **ヘッダー**は「名前: 値」の行の並びです。**空行**（何も書いていない1行）で、ヘッダーが終わったことを知らせます。
- **本文**は、空行の後ろです。どこまでが本文かは、ヘッダーの `Content-Length`（本文の**バイト数**）で伝えます（エラー1）。

状態コードは、最初の数字で、おおまかな種類が分かります。



図6-3: 状態コードの種類。最初の数字で、うまくいった（2）・別の場所へ（3）・頼み方が違う（4）・サーバーの失敗（5）が分かる

まず動かす最小コード

例1-1: 届いた要求を、そのまま表示するサーバー

> **▼ このコードがやること（先に日本語で）：** `127.0.0.1:3000` で接続を待ちます。接続が来たら、要求の1行目とヘッダーを空行まで1行ずつ読み、`Content-Length` があれば、その長さだけ本文も読みます。読んだ要求をそのまま画面に表示し、「こんにちは、HTTP！」という文字を、HTTP の応答の形にして返します。外のクレートは使いません。

```rust
//! HTTP の要求と応答が「ただの文字」であることを確かめる、とても小さなサーバー。
//! 届いた要求をそのまま表示し、決まった文字を返す（標準ライブラリだけで書く）。
//! 実行: cargo run --bin u01_raw

use std::io::{BufRead, BufReader, Read, Write};
use std::net::{TcpListener, TcpStream};

/// 1つの接続を処理する: 要求を読んで表示し、応答を返す
fn handle(mut stream: TcpStream) -> std::io::Result<()> {
    let mut reader = BufReader::new(&stream);

    // 1行目（要求行）とヘッダーを、空行が来るまで1行ずつ読む
    let mut head = Vec::new();
    loop {
        let mut line = String::new();
        reader.read_line(&mut line)?;
        let line = line.trim_end().to_string();
        if line.is_empty() {
            break;
        }
        head.push(line);
    }

    // Content-Length のヘッダーがあれば、その長さ（バイト数）だけ本文を読む
    let mut length = 0;
    for line in &head {
        if let Some((name, value)) = line.split_once(':')
            && name.eq_ignore_ascii_case("content-length")
        {
            length = value.trim().parse().unwrap_or(0);
        }
    }
    let mut body = vec![0; length];
    reader.read_exact(&mut body)?;

    println!("--- 届いた要求 ---");
    for line in &head {
        println!("{line}");
    }
    if !body.is_empty() {
        println!("（空行）");
        println!("{}", String::from_utf8_lossy(&body));
    }

    // 応答を「文字列」として組み立てて送り返す。Content-Length は本文のバイト数
    let reply = "こんにちは、HTTP！\n";
    let response = format!(
        "HTTP/1.1 200 OK\r\nContent-Type: text/plain; charset=utf-8\r\nContent-Length:
{}\r\n\r\n{}",
```

```
        reply.len(),
        reply
    );
    stream.write_all(response.as_bytes())?;
    Ok(())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:3000")?;
    println!("http://127.0.0.1:3000 で待っています（Ctrl+C で終了）");
    // 接続が来るたびに、1つずつ処理する
    for stream in listener.incoming() {
        handle(stream?)?;
    }
    Ok(())
}
```

練習用のプロジェクトの `src/bin/u01_raw.rs` として保存し、1つ目の PowerShell の窓で起動します。

```
cargo run --bin u01_raw
```

2つ目の PowerShell の窓で、`curl.exe` を使って、GET の要求と POST の要求を1回ずつ送ります。`-i` は、応答の1行目とヘッダーも表示する指定、`-d` は、本文にフォームの値を入れて POST で送る指定です。

```
curl.exe -i "http://127.0.0.1:3000/hello?name=ada"
curl.exe -i -d "title=shopping&body=milk" http://127.0.0.1:3000/notes
```

▼ 実行結果（curl の窓）

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 26

こんにちは、HTTP！
```

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 26

こんにちは、HTTP！
```

▼ 実行結果（サーバーの窓。3つ目の要求は、問3の `curl.exe -v` で送ったものです）

```
http://127.0.0.1:3000 で待っています（Ctrl+C で終了）
--- 届いた要求 ---
GET /hello?name=ada HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: curl/8.21.0
Accept: */*
--- 届いた要求 ---
POST /notes HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: curl/8.21.0
Accept: */*
Content-Length: 24
Content-Type: application/x-www-form-urlencoded
（空行）
title=shopping&body=milk
--- 届いた要求 ---
GET /hello?name=ada HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: curl/8.21.0
Accept: */*
```

> **注意:** PowerShell 5.1 で、curl の窓の日本語が `縺薙ｓ縺ｫ縺。縺ｯ` のように化けるときは、最初の `curl.exe` の前に `[Console]::OutputEncoding = [System.Text.Encoding]::UTF8` を実行します。サーバーが返した UTF-8 の文字を、PowerShell が別の文字コードで読んでいるのが原因です。

▼ コードを1つずつ分解して解説

解説1: 要求は、ただの文字の並び

- サーバーの窓には、curl が送ってきた文字が、そのまま表示されました。1行目の `GET /hello?name=ada HTTP/1.1` が要求行で、その下の `Host:`・`User-Agent:`・`Accept:` がヘッダーです。curl は、何も指定しなくても、この3つのヘッダーを付けて送ります。
- URL の `?name=ada` の部分（クエリ）は、パスの一部として要求行に入っています。
- POST のほうには、`Content-Length: 24` と `Content-Type: application/x-www-form-urlencoded`（フォームの形の本文です）が加わり、空行の後ろに、本文の `title=shopping&body=milk` が届きました。24 は、この本文のバイト数です。

解説2: 応答も、自分で書いた文字

- curl の窓の `HTTP/1.1 200 OK` から `こんにちは、HTTP！` までは、`format!` で作った文字列そのものです。行の終わりは `\r\n`（改行の前に「行の先頭へ戻る」を付けた2文字）で、HTTP の決まりです。
- このサーバーは、どんなパスやメソッドが来ても、同じ応答を返します。`/notes` に POST しても、何も保存していません。「どのパスに、どう答えるか」を決めるのが、次の例1-2と、UNIT 02 から使うフレームワーク（Topcoat）の役目です。

解説3: TcpListener は、接続を1つずつ受け取る

- `TcpListener::bind("127.0.0.1:3000")` は、「この PC の 3000 番の窓口で、接続を待つ」という意味です。`127.0.0.1` は「この PC 自身」を表す特別なアドレスで、ほかの PC からはつながりません。
- `listener.incoming()` は、接続が来るたびに1つずつ取り出します。このサーバーは、1つの接続を処理し終えるまで、次の接続を受け付けません。たくさんの人が同時に使うサーバーでは、async（第5巻）で同時にさばきます。Topcoat は、その部分を引き受けてくれます。

もう一歩: 実務に近い形

例1-2: メソッドとパスで、返す状態コードを変える

> **▼ このコードがやること（先に日本語で）**: 要求行を空白で区切って、メソッドとパスを取り出します。その組み合わせで、200（トップページ）・405（使えないメソッド）・403（見られないページ）・301（引っ越したページ）・303（伝言を受け取ったので、一覧を見てください）・404（無いページ）を返し分けます。`POST /messages` で受け取った伝言は、プログラムの中の `Vec` に貯め、`GET /messages` で一覧にします。

```rust
//! 要求のメソッドとパスを見て、いろいろな状態コードを返すサーバー（標準ライブラリだけで書く）。
//! 実行: cargo run --bin u01_status

use std::io::{BufRead, BufReader, Read, Write};
use std::net::{TcpListener, TcpStream};

/// 届いた要求（要求行・本文）
struct Request {
    method: String,
    path: String,
    body: String,
}

/// 要求を読む: 1行目（要求行）・ヘッダー・本文
fn read_request(stream: &TcpStream) -> std::io::Result<Request> {
    let mut reader = BufReader::new(stream);
    let mut first = String::new();
    reader.read_line(&mut first)?;
    // 「GET /messages HTTP/1.1」を空白で区切る
    let mut parts = first.split_whitespace();
    let method = parts.next().unwrap_or("").to_string();
    let path = parts.next().unwrap_or("").to_string();

    let mut length = 0;
    loop {
        let mut line = String::new();
        reader.read_line(&mut line)?;
        let line = line.trim_end();
        if line.is_empty() {
            break;
        }
        if let Some((name, value)) = line.split_once(':')
            && name.eq_ignore_ascii_case("content-length")
        {
            length = value.trim().parse().unwrap_or(0);
        }
    }
    let mut body = vec![0; length];
    reader.read_exact(&mut body)?;
    let body = String::from_utf8_lossy(&body).to_string();
    Ok(Request { method, path, body })
}

/// 応答を組み立てて送る。HEAD のときは、ヘッダーだけを送る
fn respond(
    stream: &mut TcpStream,
    request: &Request,
    status: &str,
    headers: &[(&str, &str)],
```

```
    body: &str,
) -> std::io::Result<()> {
    let mut text = format!(
        "HTTP/1.1 {status}\r\nContent-Type: text/plain; charset=utf-8\r\nContent-
Length: {}\r\n",
        body.len()
    );
    for (name, value) in headers {
        text.push_str(&format!("{name}: {value}\r\n"));
    }
    text.push_str("\r\n");
    if request.method != "HEAD" {
        text.push_str(body);
    }
    stream.write_all(text.as_bytes())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:3000")?;
    println!("http://127.0.0.1:3000 で待っています（Ctrl+C で終了）");
    // POST /messages で受け取った伝言を、ここに貯める
    let mut messages: Vec<String> = Vec::new();

    for stream in listener.incoming() {
        let mut stream = stream?;
        let request = read_request(&stream)?;
        println!("{} {}", request.method, request.path);

        match (request.method.as_str(), request.path.as_str()) {
            // 見るだけの操作（GET と HEAD）
            ("GET" | "HEAD", "/") => {
                respond(&mut stream, &request, "200 OK", &[], "トップページです\n")?
            }
            // / は GET と HEAD でしか使えない
            (_, "/") => respond(
                &mut stream,
                &request,
                "405 Method Not Allowed",
                &[("Allow", "GET, HEAD")],
                "その方法は使えません\n",
            )?,
            ("GET", "/admin") => respond(
                &mut stream,
                &request,
                "403 Forbidden",
                &[],
                "ここは見られません\n",
            )?,
            // 引っ越したページ: 新しい場所を Location で教える
            ("GET", "/old-page") => respond(
```

```
                &mut stream,
                &request,
                "301 Moved Permanently",
                &[("Location", "/")],
                "",
            )?,
            // 伝言を受け取ったら、303 で「GET /messages を見てください」と返す
            ("POST", "/messages") => {
                let text = request.body.split_once('=').map(|(_, value)| value);
                messages.push(text.unwrap_or("").to_string());
                respond(
                    &mut stream,
                    &request,
                    "303 See Other",
                    &[("Location", "/messages")],
                    "",
                )?
            }
            ("GET", "/messages") => {
                let list = format!("伝言は {} 件: {:?}\n", messages.len(), messages);
                respond(&mut stream, &request, "200 OK", &[], &list)?
            }
            _ => respond(
                &mut stream,
                &request,
                "404 Not Found",
                &[],
                "ページが見つかりません\n",
            )?,
        }
    }
    Ok(())
}
```

`src/bin/u01_status.rs` として保存し、例1-1のサーバーを Ctrl+C で止めてから起動します。2つ目の窓から、いろいろな要求を送ります。`-X DELETE` はメソッドを DELETE にする指定、`-I` は HEAD（ヘッダーだけをください）で送る指定、`-L` は 3xx の応答が来たら `Location` の場所へ自動でたどる指定です。

```
cargo run --bin u01_status
```

▼ 実行結果（curl の窓。`###` の行は、送ったコマンドです）

```
### curl.exe -i http://127.0.0.1:3000/
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 25

トップページです

### curl.exe -i http://127.0.0.1:3000/nothing
HTTP/1.1 404 Not Found
Content-Type: text/plain; charset=utf-8
Content-Length: 34

ページが見つかりません

### curl.exe -i http://127.0.0.1:3000/admin
HTTP/1.1 403 Forbidden
Content-Type: text/plain; charset=utf-8
Content-Length: 28

ここは見られません

### curl.exe -i http://127.0.0.1:3000/old-page
HTTP/1.1 301 Moved Permanently
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Location: /


### curl.exe -i -X DELETE http://127.0.0.1:3000/
HTTP/1.1 405 Method Not Allowed
Content-Type: text/plain; charset=utf-8
Content-Length: 31
Allow: GET, HEAD

その方法は使えません

### curl.exe -I http://127.0.0.1:3000/
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 25

### curl.exe -i -d message=hello http://127.0.0.1:3000/messages
HTTP/1.1 303 See Other
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Location: /messages


### curl.exe -i -L -d message=world http://127.0.0.1:3000/messages
```

```
HTTP/1.1 303 See Other
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Location: /messages

HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 36

伝言は 2 件: ["hello", "world"]
```

▼ 実行結果（サーバーの窓）

```
http://127.0.0.1:3000 で待っています（Ctrl+C で終了）
GET /
GET /nothing
GET /admin
GET /old-page
DELETE /
HEAD /
POST /messages
POST /messages
GET /messages
```

▼ コードを1つずつ分解して解説

解説1: 404・403・405 は、それぞれ違う「断り方」

- `/nothing` は、そもそも無いページなので **404 Not Found** です。`/admin` は、ページはあるが見せられないので **403 Forbidden** です。
- `DELETE /` は、ページはあるが、その**メソッド**では使えないので **405 Method Not Allowed** です。405 のときは、`Allow: GET, HEAD` のヘッダーで「使えるメソッドはこれです」と教えるのが HTTP の決まりです。
- どれも「頼んだ側を直せば通る」4xx の仲間ですが、直し方が違います（404 は URL を、403 は権限を、405 はメソッドを直す）。だから、使い分けると、使う人に正しい直し方が伝わります。

解説2: 301 と 303 は、Location で行き先を教える

- `/old-page` は **301 Moved Permanently** と `Location: /` を返しました。「このページは、ずっと `/` に引っ越しました」という意味です。ブラウザや検索エンジンは、次からは新しい場所を使います。
- `POST /messages` は **303 See Other** と `Location: /messages` を返しました。「受け取りました。結果は `GET /messages` で見てください」という意味です。`-L` を付けた最後のコマンドでは、curl が `Location` をたどって、自動で `GET /messages` を送り、`伝言は 2 件: ["hello", "world"]` を受け取りました。サーバーの窓にも、`POST /messages` の後に `GET /messages` が届いています。

解説3: POST の後に 303 で GET へ移す（PRG）

- もし `POST /messages` が、303 ではなく 200 で一覧を返していたら、ブラウザの画面は「POST の結果」のままです。利用者が再読み込み（F5）を押すと、ブラウザは**同じ POST をもう一度送り**、伝言が二重に保存されます。
- 303 で GET の画面へ移しておけば、再読み込みで送り直されるのは `GET /messages`（見るだけ）なので、何も増えません。この「POST → Redirect（303）→ GET」の形を、頭文字をとって **PRG** と呼びます。RustNote のフォームも、すべてこの形です（第9巻 UNIT 02）。



図6-4: PRG（POST → Redirect → GET）の流れ。POST の返事を 303 にして GET の画面へ移すので、再読み込みで送り直されるのは見るだけの GET になる

- GET は「見るだけ」で、何回送っても DB を変えない（**安全**な）メソッドとして扱います。ブラウザは、リンクの先を前もって読み込んだり、再読み込みで送り直したりするので、GET で何かを変えると、利用者が知らないうちに変わってしまいます。

解説4: HEAD は、ヘッダーだけを返す

- `curl.exe -I`（HEAD）には、`200 OK` と `Content-Length: 25` だけが返り、本文はありませんでした。`respond` の中の `if request.method != "HEAD"` で、本文を付けないようにしているからです。HEAD は「本文の大きさや種類だけを、先に知りたい」ときに使います。

よくあるエラーと読み方

エラー1（エラーが出ない間違い）: Content-Length に文字数を入れた

例1-1の `reply.len()`（バイト数）を、`reply.chars().count()`（文字数）にしてしまった場合です（`src/bin/u01_len_bug.rs`）。

```rust
//! エラー1の例: Content-Length に、バイト数ではなく文字数を入れてしまったサーバー。
//! 実行: cargo run --bin u01_len_bug

use std::io::{BufRead, BufReader, Write};
use std::net::TcpListener;

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:3000")?;
    println!("http://127.0.0.1:3000 で待っています（Ctrl+C で終了）");
    for stream in listener.incoming() {
        let mut stream = stream?;
        // 要求の1行目とヘッダーを、空行まで読み飛ばす
        let mut reader = BufReader::new(&stream);
        loop {
            let mut line = String::new();
            reader.read_line(&mut line)?;
            if line.trim_end().is_empty() {
                break;
            }
        }

        let reply = "こんにちは、HTTP！\n";
        // 間違い: chars().count() は「文字数」。Content-Length に入れるのは「バイト数」
        let response = format!(
            "HTTP/1.1 200 OK\r\nContent-Type: text/plain; charset=utf-8\r\nContent-
Length: {}\r\n\r\n{}",
            reply.chars().count(),
            reply
        );
        stream.write_all(response.as_bytes())?;
    }
    Ok(())
}
```

```
cargo run --bin u01_len_bug
```

別の窓から:

```
curl.exe -i http://127.0.0.1:3000/
```

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 12

こんにち
```

- **何が起きたか:** 本文が「こんにち」で切れました。エラーは、どこにも出ていません。`Content-Length: 12` は、`こんにちは、HTTP！\n` の**文字数**（12 文字）です。ところが、日本語の1文字は UTF-8 で3バイトなので、本当の本文は 26 バイトあります。curl は、`Content-Length` を信じて 12 バイトだけを本文として読み、残りを捨てました。12 バイトは、日本語4文字ぶん（こ・ん・に・ち）です。
- **英語だけで試すと気づかない:** 英数字は1文字1バイトなので、文字数とバイト数が同じになり、この間違いは見えません。日本語のアプリでは、日本語で試すことが大切です（第2巻 UNIT 05、第9巻 UNIT 03）。
- **直し方:** `Content-Length` には `reply.len()`（`String` の `len()` はバイト数）を入れます。UNIT 02 から使う Topcoat は、この長さを自動で正しく計算してくれます。

エラー2: 誰も待っていないポートに送った（curl: (7)）

サーバーを止めたまま、`curl.exe` で要求を送った場合です。

```
curl.exe -i http://127.0.0.1:3000/
```

```
curl: (7) Failed to connect to 127.0.0.1:3000 after 2029 ms: Could not connect to
server
```

- **何が起きたか:** 「127.0.0.1 の 3000 番に、約2秒試しても接続できませんでした」というエラーです。`(7)` は curl のエラーの番号で、「接続できなかった」を表します。HTTP の状態コード（404 など）は出ていません。状態コードは、サーバーが応答を返したときにだけ届くものだからです。
- **確かめること:** サーバーの窓で、`http://127.0.0.1:3000 で待っています` が表示されているか、ポートの番号が同じかを確かめます。サーバーが起動の途中でエラーになって止まっていることもあります。
- **404 との違い:** 404 は「サーバーは動いていて、そのページが無い」、curl の `(7)` は「サーバーにたどり着けない」です。どちらの段階で失敗しているかを、最初に見分けます。

エラー3: 同じポートで2つ起動した（os error 10048）

例1-1のサーバーを止めずに、別の窓から例1-2のサーバーを起動すると、`TcpListener::bind` が失敗します。Windows では `os error 10048`（そのアドレスとポートは、もう使われています）、Linux では `os error 98` のエラーになります。先に動いているサーバーを Ctrl+C で止めてから、起動し直します（RustNote で同じエラーを見た例は、第12巻 UNIT 01 のエラー2にあります）。

RustNote ではここで使う

RustNote は、例1-2のような「メソッドとパスで返事を分ける」処理を、Topcoat に任せています。RustNote の `src/routes.rs` の先頭のコメントには、ファイルと URL の対応の表があります（抜き出しています）。

```
//! | ファイル | URL |
//! |---|---|
//! | routes.rs                | GET /（トップ）、全ページ共通のレイアウト、404 の受け皿 |
//! | routes/health.rs         | GET /health |
//! | routes/signup.rs         | GET /signup・POST /signup |
//! | routes/login.rs          | GET /login・POST /login |
//! | routes/logout.rs         | POST /logout |
//! | routes/notes.rs          | GET /notes（?q= 検索・?page= ページ送り）・POST /notes |
```

- 例1-2の `match (method, path)` の1行ずつが、RustNote では1つの関数（`#[page]` や `#[page(POST)]` を付けたもの）になります。無いページの 404、使えないメソッドの 405 は、Topcoat が自動で返します。
- RustNote のフォームの POST は、成功すると必ず 303 で GET の画面へ移します（PRG）。たとえば、ノートの作成（`POST /notes`）は、303 で、作ったノートの詳細（`GET /notes/{id}`）へ移ります。

RustNote が返す状態コードの決まりは、`src/error.rs` の先頭のコメントにまとめてあります（抜き出しています）。

```
//! | AppError | ステータス | 起きる場面 |
//! |---|---|---|
//! | `InvalidTitle` など入力の検証エラー | 400 | 空のタイトル、形式の違うメール |
//! | `Unauthorized` | 401 | ログインしていない（通常はログイン画面へのリダイレクトで扱う） |
//! | `NotFound` | 404 | 存在しないノート、**他人のノート**（あること自体を隠す） |
//! | `EmailTaken` / `Conflict` | 409 | 登録済みのメール、ほかの画面で先に更新された |
//! | `Internal` | 500 | DB の故障など。画面には詳細を出さず、ログに原因を書く |
```

- 例1-2では出てこなかった **400**（入力の形が違う）・**401**（ログインが必要）・**409**（ほかの更新とぶつかった）・**500**（サーバーの中の失敗）も使っています。他人のノートを 403 ではなく 404 にしている理由は、第8巻 UNIT 01 で扱いました。

練習問題

問1: 例1-2のサーバーに、`GET /health` を足してください。`200 OK` で、本文は `{"status":"ok"}`、`Content-Type` は `application/json`（JSON の形の本文です）にします。足したら、`GET /health` と `POST /health` を送って、それぞれの応答を確かめてください。`POST /health` の応答は、HTTP の決まりとして正しいでしょうか。

問2: 「伝言を `GET /messages?add=hello` で足せる」ように作ったとします。この作りには、どんな困ったことが起きるでしょうか。解説3の内容をもとに、2つ挙げてください。

問3: 例1-1で取った `curl.exe -v` の表示（下）の中から、curl が送った要求の部分と、サーバーが返した応答の部分を見分けてください。それぞれ、どの記号で始まっていますか。また、`*` で始まる行は何を表しているでしょうか。

```
*   Trying 127.0.0.1:3000...
* Established connection to 127.0.0.1 (127.0.0.1 port 3000) from 127.0.0.1 port 55388
* using HTTP/1.x
> GET /hello?name=ada HTTP/1.1
> Host: 127.0.0.1:3000
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/plain; charset=utf-8
< Content-Length: 26
<
{ [26 bytes data]
* Connection #0 to host 127.0.0.1:3000 left intact
```

解答と解説

問1の解答:

`match` の、`// 見るだけの操作（GET と HEAD）` の行の前に、次の腕を足します（足したものを `src/bin/u01_q1.rs` として保存しました）。`respond` は `Content-Type` を `text/plain` に決めているので、この腕では、応答の文字を自分で組み立てます。

```
            // 問1: 動いているかを外から確かめる URL。JSON で返すので Content-Type も変える
            ("GET", "/health") => {
                let body = r#"{"status":"ok"}"#;
                let text = format!(
                    "HTTP/1.1 200 OK\r\nContent-Type: application/json\r\nContent-
Length: {}\r\n\r\n{body}",
                    body.len()
                );
                stream.write_all(text.as_bytes())?
            }
```

```
cargo run --bin u01_q1
```

```
### curl.exe -i http://127.0.0.1:3000/health
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 15

{"status":"ok"}

### curl.exe -i -X POST http://127.0.0.1:3000/health
HTTP/1.1 404 Not Found
Content-Type: text/plain; charset=utf-8
Content-Length: 34

ページが見つかりません
```

- `GET /health` には、`Content-Type: application/json` と `{"status":"ok"}` が返りました。`Content-Length: 15` は、`{"status":"ok"}` のバイト数です。`r#"..."#` は、中に `"` を含む文字列を、`\"` と書かずにそのまま書ける Rust の書き方（生の文字列リテラル）です。
- `POST /health` には **404** が返りました。`match` のどの腕にも当たらず、最後の `_` に落ちたからです。HTTP の決まりとしては、`/health` というページはあるので、**405 Method Not Allowed** と `Allow: GET` を返すのが正確です。例1-2の `/` と同じように、`(_, "/health")` の腕を足せば直せます。こうした決まりを、ページごとに自分で書き続けるのは大変です。Topcoat は、ページに付けたメソッドから、405 と `Allow` を自動で返してくれます（第9巻 UNIT 02 で、RustNote の 405 を確かめました）。
- RustNote の `GET /health` も、同じ `{"status":"ok"}` を返します。公開したあと、外から「動いているか」を確かめるのに使います（第12巻 UNIT 01）。

問2の解答:

1. **再読み込みで、伝言が増える:** `GET /messages?add=hello` の画面で再読み込みを押すと、ブラウザは同じ GET をもう一度送るので、伝言がもう1件増えます。GET は「何回送っても変わらない」ことが前提なので、ブラウザは確かめずに送り直します。
2. **リンクを踏んだだけで、伝言が増える:** GET は、リンク（`<a href="...">`）や画像（`<img src="...">`）でも送られます。ほかのサイトに `<img src="http://127.0.0.1:3000/messages?add=spam">` と書かれていたら、そのページを開いただけで、利用者のブラウザが伝言を足してしまいます。ブラウザが「先読み」（リンクの先を前もって読み込むこと）で送ることもあります。

- だから、何かを**変える**操作は POST（や PUT・DELETE）にして、GET は**見るだけ**にします。POST にしても、ほかのサイトからフォームで送らせる攻撃（CSRF）は残るので、その守りを足します（第8巻 UNIT 01、この巻の UNIT 10（準備中））。

問3の解答:

- `>` **で始まる行**が、curl が送った要求です（`> GET /hello?name=ada HTTP/1.1` から、空の `>` の行まで）。`<` **で始まる行**が、サーバーが返した応答の1行目とヘッダーです（`< HTTP/1.1 200 OK` から、空の `<` の行まで）。矢印の向き（`>` は外へ、`<` は内へ）と覚えます。
- `*` **で始まる行**は、curl が自分の動きを説明しているメモです。`Trying 127.0.0.1:3000...`（つなごうとしています）、`Established connection`（つながりました）、`Request completely sent off`（要求を送り終えました）、`Connection #0 to host 127.0.0.1:3000 left intact`（接続は閉じずに残しました）のように、HTTP の文字ではなく、その外側で起きたことを表します。
- `{ [26 bytes data]` は、本文を 26 バイト受け取ったという印です。`-v` と一緒に `-o NUL`（本文は捨てる）を付けたので、本文そのものは表示されていません。

まとめ

- HTTP の要求と応答は、どちらも「1行目・ヘッダー・空行・本文」の文字の並び。要求の1行目はメソッドとパス、応答の1行目は状態コード。本文の長さは `Content-Length` に**バイト数**で書く。
- 状態コードは、2xx（うまくいった）・3xx（別の場所へ。`Location` で行き先を教える）・4xx（頼み方が違う。404・403・405 は直し方が違う）・5xx（サーバーの失敗）。curl の `(7)` は、そもそもサーバーにたどり着けていない。
- GET は見るだけ、POST は変える。POST が成功したら 303 で GET の画面へ移す（PRG）と、再読み込みで二重に保存されない。

この巻のまとめ

この版に収録した UNIT 01 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: HTTP の要求・応答・状態コード

- ☐ HTTP の要求と応答が「1行目・ヘッダー・空行・本文」の文字の並びであることを、標準ライブラリだけのサーバーで確かめられる
- ☐ 要求の1行目（メソッド・パス・版）と、応答の1行目（版・状態コード・説明）を読める
- ☐ `Content-Length` に入れるのが文字数ではなくバイト数である理由と、まちがえたときに本文が切れることを説明できる
- ☐ 200・301・303・403・404・405 の違いと、405 のときに `Allow` を付ける理由を言える
- ☐ GET は見るだけ、POST は変えるという使い分けと、POST の後に 303 で GET へ移す PRG の理由を説明できる
- ☐ `curl.exe` の `-i`・`-d`・`-X`・`-I`・`-L`・`-v` を使い分け、`-v` の `>`・`<`・`*` の行を読み分けられる
- ☐ curl の `(7)`（たどり着けない）と、404（たどり着いたが、ページが無い）を見分けられる

> **お知らせ:** この版には UNIT 01 を収録しています。UNIT 02（Topcoat 0.10 の最小のアプリ）・UNIT 03（ルーター・page・route・layout）・UNIT 04（URL の部品と Cx）・UNIT 05（view!・コンポーネント・HTML のエスケープ）・UNIT 06（フォーム・JSON・PRG）・UNIT 07（app_context・memoize）・UNIT 08（アセット・CSS・束）・UNIT 09（runtime・signal・procedure・shard）・UNIT 10（エラー・クッキー・セッション・CSRF）は準備中で、次の更新から順に追加します。それまでの間、Topcoat を使った完成したアプリの例は、教材のフォルダの `Rust/rustnote/src/routes.rs` から読み始められます。

用語ミニ辞典

この巻（UNIT 01）に出てきた用語です。

UNIT 01 の用語

用語	意味
HTTP	ブラウザとサーバーが、要求と応答をやり取りするときの決まり
要求（request）	サーバーへの頼み。1行目（メソッド・パス・版）・ヘッダー・空行・本文でできている
応答（response）	サーバーからの返事。1行目（版・状態コード・説明）・ヘッダー・空行・本文でできている
メソッド	要求の種類。GET（見る）・POST（変える）・HEAD（ヘッダーだけ見る）・DELETE（消す）など
パス	要求の1行目の、`/notes` のような場所の部分。`?` の後ろはクエリ
ヘッダー	「名前: 値」の行の並び。空行で終わる
`Content-Length`	本文のバイト数。受け取る側は、この数だけ本文を読む
状態コード	応答の1行目の3桁の数。2xx 成功・3xx 別の場所へ・4xx 頼み方が違う・5xx サーバーの失敗
`Location`	3xx の応答で、行き先を教えるヘッダー
`Allow`	405 の応答で、使えるメソッドを教えるヘッダー
PRG	POST → Redirect（303）→ GET。再読み込みで二重に送られないようにする形
安全なメソッド	何回送っても、サーバーの中のデータを変えないメソッド（GET・HEAD）
`TcpListener`	決まったアドレスとポートで、接続を待つ標準ライブラリの型
`127.0.0.1`	「この PC 自身」を表すアドレス。ほかの PC からはつながらない
curl の `(7)`	サーバーに接続できなかったときの curl のエラー